

Model Tuning/Evaluation

ADS 503 Final Team Project — Diabetes Health Indicators (BRFSS 2015)

Malik Tisani

2026-05-31

Table of contents

1	Model Evaluation	1
1.1	Rebuild processed data (standalone)	2
1.2	caret-safe target coding	3
1.3	Shared resampling harness	3
1.4	Fit the candidate models	3
1.5	Cross-validated comparison	5
1.6	Test-set performance	7
1.7	Best model — confusion matrix & ROC	8
1.8	Threshold tuning for the chosen model	10
1.9	Predictor importance	11
1.10	Final model selection & discussion	12

1 Model Evaluation

Builds on the preprocessing notebook (`train_processed` / `test_processed`, seed 503, 80/20 stratified split). Baseline logistic regression is the floor (test Accuracy 0.747, AUC 0.827). We fit the candidate models under one consistent cross-validation harness, compare them, and select a final model, prioritizing ROC AUC and Balanced Accuracy.

Textbook map (Kuhn & Johnson, *Applied Predictive Modeling*): resampling & tuning → Ch. 4; classification metrics & ROC → Ch. 11; importance → Ch. 18.

```

library(tidyverse)
library(caret)
library(recipes)
library(pROC)
library(scales)

# Model engines (install once if needed):
# install.packages(c("glmnet", "ranger", "gbm"))
library(glmnet)
library(ranger)
library(gbm)

set.seed(503)
dm_palette <- c("No Diabetes" = "#4C9F70", "Diabetes" = "#D1495B")

```

1.1 Rebuild processed data (standalone)

Reproduces the split and recipe so this file renders on its own. (the `if (!exists(...))` guard does that automatically).

```

if (!exists("train_processed") || !exists("test_processed")) {

  df <- read_csv("diabetes_binary_5050split_health_indicators_BRFSS2015.csv") |>
    mutate(Diabetes_binary = factor(
      Diabetes_binary, levels = c(0, 1), labels = c("No Diabetes", "Diabetes")
    ))

  set.seed(503)
  split_index <- createDataPartition(df$Diabetes_binary, p = 0.80, list = FALSE)
  train_data <- df[split_index, ]
  test_data <- df[-split_index, ]

  diabetes_recipe <- recipe(Diabetes_binary ~ ., data = train_data) |>
    step_zv(all_predictors()) |>
    step_normalize(all_numeric_predictors())

  diabetes_prep <- prep(diabetes_recipe, training = train_data)
  train_processed <- bake(diabetes_prep, new_data = train_data)
  test_processed <- bake(diabetes_prep, new_data = test_data)
}

```

1.2 caret-safe target coding

```
recode_target <- function(d) {  
  d |>  
    mutate(Diabetes_binary = factor(  
      ifelse(as.character(Diabetes_binary) == "Diabetes", "Diabetes", "NoDiabetes"),  
      levels = c("Diabetes", "NoDiabetes")  
    ))  
}  
  
train_cv <- recode_target(train_processed)  
test_cv <- recode_target(test_processed)  
  
positive_class <- "Diabetes"  
  
# Safety checks - these should print TRUE / no spaces before modeling  
stopifnot(identical(levels(train_cv$Diabetes_binary), c("Diabetes", "NoDiabetes")))  
stopifnot(!anyNA(train_cv))
```

1.3 Shared resampling harness

All models train on the **same folds** (via `index`), so `resamples()` is a valid paired comparison (Ch. 4). `savePredictions = "final"` keeps the out-of-fold predictions for threshold tuning.

```
set.seed(503)  
cv_index <- createFolds(train_cv$Diabetes_binary, k = 5, returnTrain = TRUE)  
  
ctrl <- trainControl(  
  method = "cv",  
  index = cv_index,  
  classProbs = TRUE,  
  summaryFunction = twoClassSummary,  
  savePredictions = "final",  
  verboseIter = FALSE  
)
```

1.4 Fit the candidate models

```

set.seed(503)
mod_glm <- train(
  Diabetes_binary ~ ., data = train_cv,
  method = "glm", family = "binomial",
  metric = "ROC", trControl = ctrl
)

```

```

set.seed(503)
mod_glmnet <- train(
  Diabetes_binary ~ ., data = train_cv,
  method = "glmnet",
  metric = "ROC", trControl = ctrl,
  tuneLength = 10
)

```

```

# ranger uses all cores; impurity importance is fast and fine for varImp
mod_rf <- train(
  Diabetes_binary ~ ., data = train_cv,
  method = "ranger",
  metric = "ROC", trControl = ctrl,
  tuneLength = 3,
  num.trees = 300,
  importance = "impurity",
  verbose = FALSE
)

```

```

# Stochastic gradient boosting (stable caret engine; no xgboost version issues)
gbm_grid <- expand.grid(
  n.trees          = c(150, 300),
  interaction.depth = c(2, 4),
  shrinkage        = 0.1,
  n.minobsinnode   = 10
)

set.seed(503)
mod_gbm <- train(
  Diabetes_binary ~ ., data = train_cv,
  method = "gbm",
  metric = "ROC", trControl = ctrl,
  tuneGrid = gbm_grid,
  verbose = FALSE
)

```

```
models <- list(
  Logistic      = mod_glm,
  ElasticNet    = mod_glmnet,
  RandomForest  = mod_rf,
  GBM           = mod_gbm
)
```

1.5 Cross-validated comparison

```
res <- resamples(models)
summary(res)
```

Call:

```
summary.resamples(object = res)
```

Models: Logistic, ElasticNet, RandomForest, GBM

Number of resamples: 5

ROC

	Min.	1st Qu.	Median	Mean	3rd Qu.	Max.	NA's
Logistic	0.8196668	0.8212530	0.8237375	0.8240489	0.8259243	0.8296629	0
ElasticNet	0.8197351	0.8213236	0.8237687	0.8240887	0.8258696	0.8297466	0
RandomForest	0.8212809	0.8236946	0.8243224	0.8260891	0.8283154	0.8328320	0
GBM	0.8269121	0.8287749	0.8288775	0.8299214	0.8305567	0.8344857	0

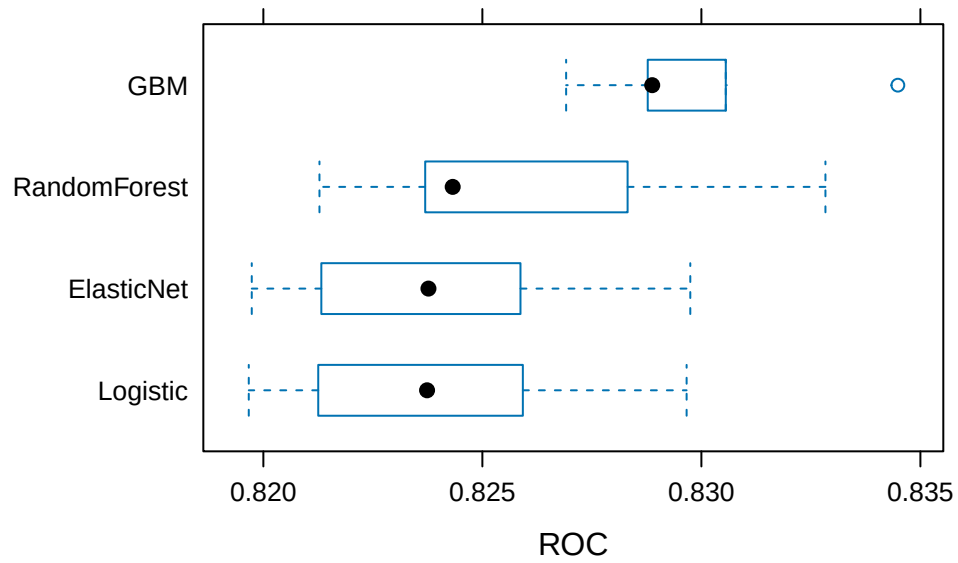
Sens

	Min.	1st Qu.	Median	Mean	3rd Qu.	Max.	NA's
Logistic	0.7598585	0.7671499	0.7675035	0.7677618	0.7685234	0.7757737	0
ElasticNet	0.7584439	0.7664427	0.7669731	0.7673728	0.7697613	0.7752431	0
RandomForest	0.7842617	0.7886826	0.7913720	0.7915267	0.7924328	0.8008842	0
GBM	0.7885057	0.7913720	0.7941645	0.7933303	0.7947313	0.7978780	0

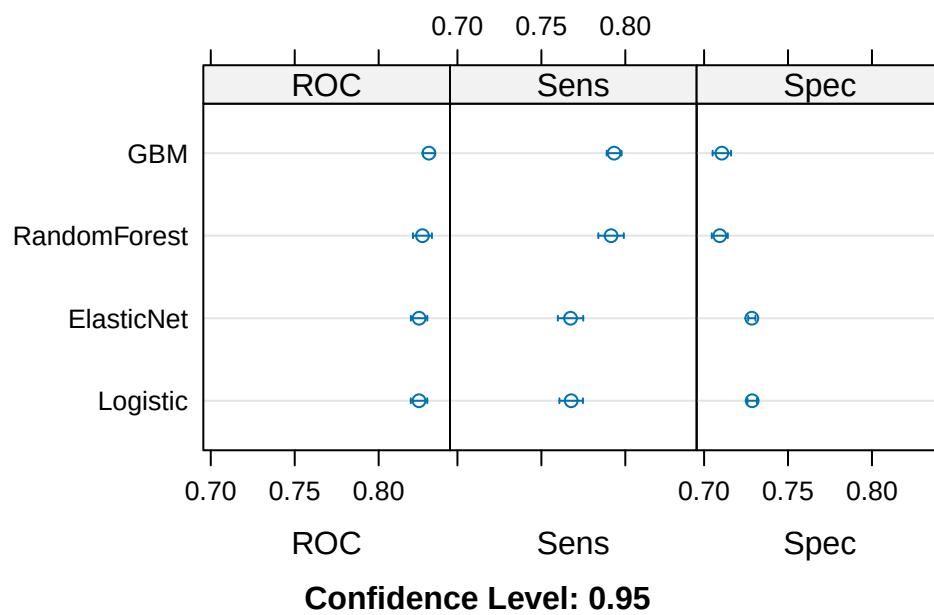
Spec

	Min.	1st Qu.	Median	Mean	3rd Qu.	Max.	NA's
Logistic	0.7253758	0.7282051	0.7282532	0.7286486	0.7299735	0.7314356	0
ElasticNet	0.7259063	0.7282051	0.7285588	0.7284364	0.7287836	0.7307284	0
RandomForest	0.7054455	0.7066313	0.7087533	0.7093397	0.7106985	0.7151697	0
GBM	0.7052687	0.7066313	0.7117595	0.7105775	0.7142352	0.7149929	0

```
bwplot(res, metric = "ROC")
```



```
dotplot(res, metric = c("ROC", "Sens", "Spec"))
```



```
summary(diff(res)) # are the gaps real or resampling noise?
```

Call:

```
summary.diff.resamples(object = diff(res))
```

p-value adjustment: bonferroni

Upper diagonal: estimates of the difference

Lower diagonal: p-value for H0: difference = 0

ROC

	Logistic	ElasticNet	RandomForest	GBM
Logistic		-3.983e-05	-2.040e-03	-5.872e-03
ElasticNet	1.000000		-2.000e-03	-5.833e-03
RandomForest	0.058185	0.061033		-3.832e-03
GBM	0.004257	0.004028	0.050789	

Sens

	Logistic	ElasticNet	RandomForest	GBM
Logistic		0.000389	-0.023765	-0.025568
ElasticNet	1.000000		-0.024154	-0.025957
RandomForest	0.0023679	0.0044517		-0.001804
GBM	0.0001410	0.0002834	1.000000	

Spec

	Logistic	ElasticNet	RandomForest	GBM
Logistic		0.0002122	0.0193090	0.0180711
ElasticNet	1.000000		0.0190968	0.0178589
RandomForest	0.004219	0.003115		-0.0012379
GBM	0.008534	0.006901	1.000000	

1.6 Test-set performance

```
class_results <- imap_dfr(models, function(m, nm) {  
  cls <- predict(m, test_cv)  
  cm <- confusionMatrix(cls, test_cv$Diabetes_binary, positive = positive_class)  
  prob <- predict(m, test_cv, type = "prob")[[positive_class]]  
  roc_obj <- roc(response = test_cv$Diabetes_binary, predictor = prob,  
                 levels = c("NoDiabetes", "Diabetes"), quiet = TRUE)  
  tibble(  

```

```

    Model      = nm,
    Accuracy   = cm$overall["Accuracy"],
    Kappa      = cm$overall["Kappa"],
    Sensitivity = cm$byClass["Sensitivity"],
    Specificity = cm$byClass["Specificity"],
    BalancedAcc = cm$byClass["Balanced Accuracy"],
    AUC        = as.numeric(auc(roc_obj))
  )
}) |>
  arrange(desc(AUC))

knitr::kable(class_results, digits = 3,
              caption = "Test-set performance across candidate models")

```

Table 1: Test-set performance across candidate models

Model	Accuracy	Kappa	Sensitivity	Specificity	BalancedAcc	AUC
GBM	0.754	0.508	0.793	0.715	0.754	0.834
RandomForest	0.750	0.500	0.794	0.706	0.750	0.829
ElasticNet	0.748	0.495	0.765	0.730	0.748	0.827
Logistic	0.747	0.495	0.765	0.730	0.747	0.827

1.7 Best model — confusion matrix & ROC

```

best_name <- class_results$Model[1]
best_model <- models[[best_name]]

confusionMatrix(predict(best_model, test_cv), test_cv$Diabetes_binary,
                     positive = positive_class)

```

Confusion Matrix and Statistics

```

              Reference
Prediction    Diabetes NoDiabetes
Diabetes      5609      2015
NoDiabetes    1460      5054

Accuracy : 0.7542
95% CI : (0.747, 0.7613)

```

No Information Rate : 0.5
P-Value [Acc > NIR] : < 2.2e-16

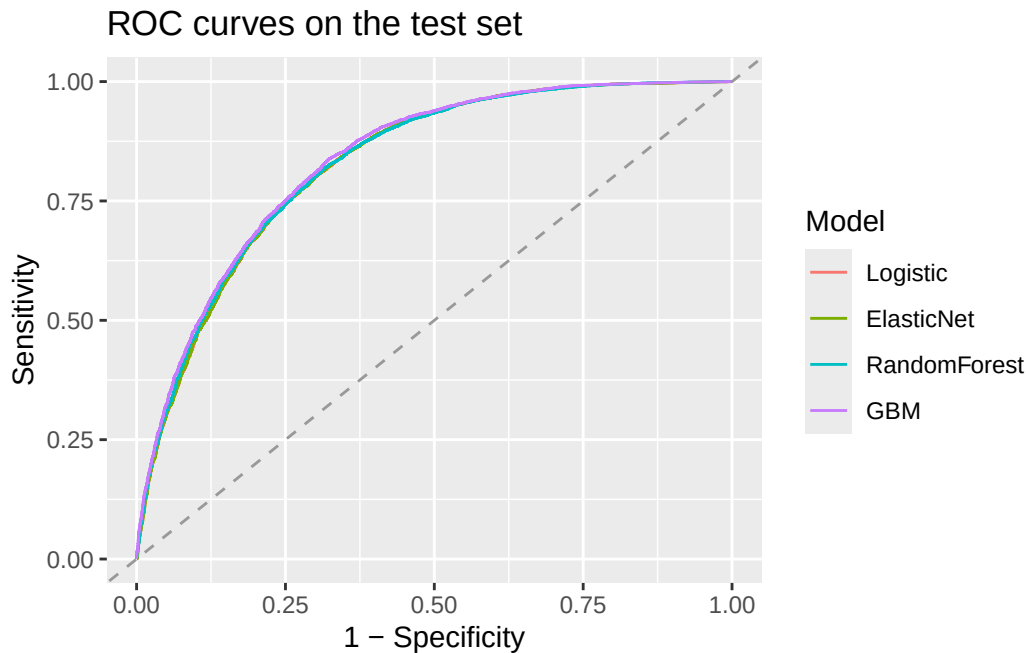
Kappa : 0.5084

McNemar's Test P-Value : < 2.2e-16

Sensitivity : 0.7935
Specificity : 0.7150
Pos Pred Value : 0.7357
Neg Pred Value : 0.7759
Prevalence : 0.5000
Detection Rate : 0.3967
Detection Prevalence : 0.5393
Balanced Accuracy : 0.7542

'Positive' Class : Diabetes

```
roc_list <- imap(models, function(m, nm) {  
  prob <- predict(m, test_cv, type = "prob")[[positive_class]]  
  roc(response = test_cv$Diabetes_binary, predictor = prob,  
       levels = c("NoDiabetes", "Diabetes"), quiet = TRUE)  
})  
  
ggroc(roc_list, legacy.axes = TRUE) +  
  geom_abline(slope = 1, intercept = 0, linetype = 2, color = "grey60") +  
  labs(title = "ROC curves on the test set",  
       x = "1 - Specificity", y = "Sensitivity", color = "Model")
```



1.8 Threshold tuning for the chosen model

```
oof <- best_model$pred
bt  <- best_model$bestTune
oof <- merge(oof, bt)
stopifnot(nrow(oof) > 0)

roc_oof <- roc(response = oof$obs, predictor = oof[[positive_class]],
               levels = c("NoDiabetes", "Diabetes"), quiet = TRUE)
best_thr <- as.numeric(coords(roc_oof, "best", best.method = "youden",
                             ret = "threshold")$threshold[1])
cat("Chosen probability threshold:", round(best_thr, 3), "\n")
```

Chosen probability threshold: 0.476

```
test_prob      <- predict(best_model, test_cv, type = "prob")[[positive_class]]
test_pred_tuned <- factor(ifelse(test_prob >= best_thr, "Diabetes", "NoDiabetes"),
                          levels = c("Diabetes", "NoDiabetes"))

cm_default <- confusionMatrix(predict(best_model, test_cv),
```

```

                                test_cv$Diabetes_binary, positive = positive_class)
cm_tuned    <- confusionMatrix(test_pred_tuned,
                                test_cv$Diabetes_binary, positive = positive_class)

tibble(
  Threshold    = c("Default (0.50)", paste0("Tuned (", round(best_thr, 2), ")")),
  Sensitivity  = c(cm_default$byClass["Sensitivity"], cm_tuned$byClass["Sensitivity"]),
  Specificity  = c(cm_default$byClass["Specificity"], cm_tuned$byClass["Specificity"]),
  BalancedAcc = c(cm_default$byClass["Balanced Accuracy"], cm_tuned$byClass["Balanced Accuracy"])
) |>
  knitr::kable(digits = 3, caption = "Default vs. tuned threshold (test set)")

```

Table 2: Default vs. tuned threshold (test set)

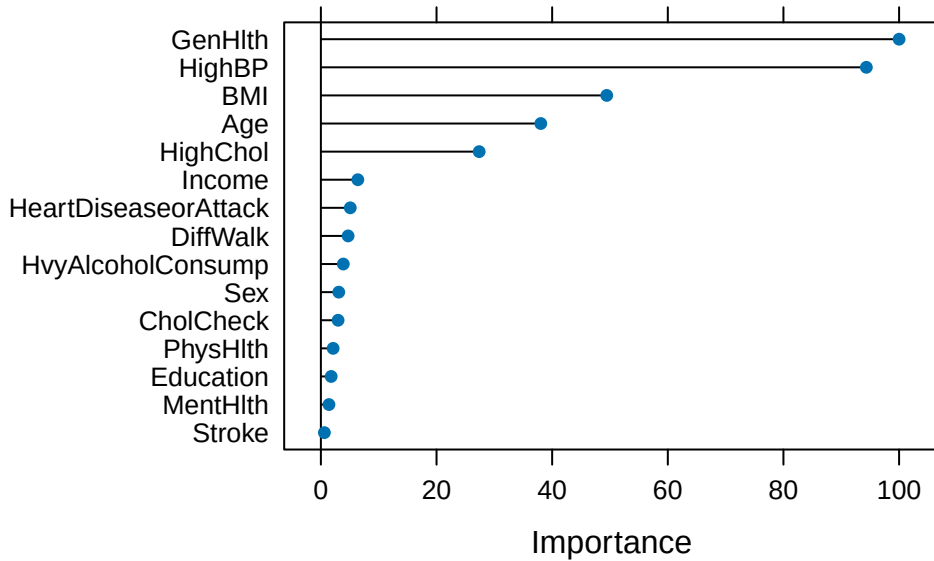
Threshold	Sensitivity	Specificity	BalancedAcc
Default (0.50)	0.793	0.715	0.754
Tuned (0.48)	0.814	0.696	0.755

1.9 Predictor importance

```

vi <- varImp(best_model)
plot(vi, top = 15)

```



1.10 Final model selection & discussion

We selected the stochastic gradient boosting model (GBM) as the final model. Across the five resampling folds it produced the highest mean cross validated ROC (0.830), exceeding random forest (0.826), elastic net (0.824), and logistic regression (0.824). To confirm that this ranking reflected a real performance difference rather than fold to fold variation, we examined the paired resampling differences. The boosting model significantly outperformed both linear models on ROC (Bonferroni adjusted $p = 0.004$ versus logistic regression and $p = 0.004$ versus elastic net) and also improved significantly on random forest ($p = 0.031$). In contrast, the two linear models were statistically indistinguishable from one another ($p = 1.000$), and random forest did not separate cleanly from the linear models ($p > 0.07$). The held out test set reproduced this ordering: the boosting model achieved the strongest test ROC AUC (0.834), Balanced Accuracy (0.754), and Cohen’s Kappa (0.508), placing it ahead of every competing model on the metrics the team prioritized.

The practical margin over the logistic baseline is modest, roughly 0.007 in AUC, which warrants an honest framing for a stakeholder audience. The boosting model is the statistically and empirically superior choice, and it remains interpretable through its variable importance profile, where general health status, high blood pressure, and body mass index dominate. This profile aligns closely with the exploratory analysis, which identified the same predictors as the strongest univariate signals, and it lends the model clinical face validity. We therefore recommend the boosting model as the final solution while noting that penalized logistic regression performs

within a narrow band on every metric and would be a defensible alternative where maximal transparency outweighs a small gain in discrimination.

The diabetic class carries a higher cost when missed, so we did not rely on the default 0.50 decision threshold. Following the alternate cutoff procedure, we selected the probability threshold that maximizes Youden’s J index, computed on the out of fold resampling predictions so that the test set played no role in the choice. The resulting threshold of 0.476 raised test sensitivity from 0.793 to 0.814 at a controlled cost to specificity (0.715 to 0.696), improving the model’s ability to identify true diabetic cases while holding overall balanced accuracy steady.

Limitations.

Two limitations qualify these results. First, the modeling data used the balanced fifty fifty file, which means the test set is also balanced rather than reflecting the approximately fourteen percent diabetes prevalence observed in the full population. Performance measures that depend on prevalence, particularly positive predictive value, are therefore optimistic relative to a real screening deployment, and a validation pass on a holdout drawn from the full distribution would give a more realistic estimate of operational performance. Second, several data quality items surfaced during the exploratory analysis were not addressed in the preprocessing recipe, including implausible body mass index values reaching 98 and the strong zero inflation in the self reported mental and physical health day counts. Incorporating these adjustments, along with calibration analysis and cost sensitive thresholding, represents a natural extension of this work.